

Chương này trình bày các giải pháp kỹ thuật và đóng góp nổi bật của đồ án trong quá trình xây dựng hệ thống. Mỗi giải pháp được trình bày theo ba phần: vấn đề đặt ra, giải pháp, và kết quả đạt được. Các nội dung về kiến trúc và thiết kế tổng thể đã trình bày ở Chương 4; chương này tập trung vào những điểm mang tính giải quyết vấn đề và đóng gói thành giải pháp có thể tái sử dụng.

## 0.1 Dựng bản đồ ba chiều chính xác từ SUMO và OpenStreetMap

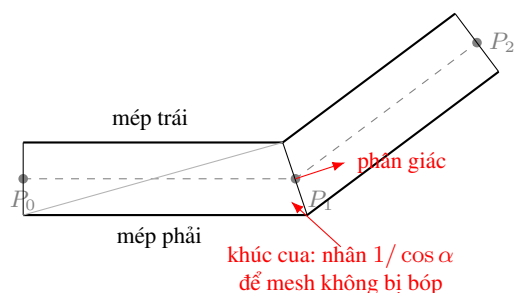
### 0.1.1 Vấn đề

Yêu cầu cốt lõi của hệ thống là dựng lại mạng đường trong không gian ba chiều bám sát dữ liệu nguồn, cho cả mạng sinh tự động lẫn bản đồ thực tế từ OpenStreetMap. Dữ liệu mạng của SUMO mô tả mỗi làn đường bằng một đường gấp khúc và mỗi nút giao bằng một đa giác hình dạng bất kỳ. Việc chuyển các mô tả này thành lưới đa giác ba chiều phát sinh nhiều khó khăn: dải mặt đường phải bám đúng đường gấp khúc và không bị bóp méo tại khúc cua; đa giác nút giao có thể lõm hoặc thậm chí tự cắt khi SUMO gộp nhiều nút sát nhau; và mặt đường của các đoạn kề nhau phải khớp nhau để xe có thể di chuyển liên tục mà không vướng.

### 0.1.2 Giải pháp

Đồ án xây dựng hai thuật toán dựng hình riêng trong Unity cho hai loại đối tượng là đoạn đường và nút giao.

**Thuật toán dựng đoạn đường.** Mỗi làn được dựng thành một dải lưới bám theo đường gấp khúc của nó. Tại mỗi đỉnh của đường gấp khúc, hệ thống tính một vector vuông góc với hướng đi để sinh ra hai đỉnh lưới ở hai mép làn. Ở các đỉnh giữa nằm trên khúc cua, vector vuông góc được lấy theo phân giác của hai đoạn kề và độ rộng được nhân thêm một hệ số theo góc cua, nhờ đó dải mặt đường không bị bóp lại ở chỗ gấp khúc. Trên cơ sở đó, hệ thống vẽ thêm vạch phân làn liên hoặc đứt đoạn và vạch biên. Để xe có thể di chuyển trong lòng đường, mỗi đoạn được sinh một tấm sàn va chạm phẳng bao phủ toàn bộ bề mặt đoạn đó. Hình 0.1 minh họa cách dựng dải mặt đường và hiệu chỉnh tại khúc cua; Giải thuật 1 tóm tắt bước sinh đỉnh lưới.



**Hình 0.1:** Dựng dải mặt đường bám đường gấp khúc và hiệu chỉnh tại khúc cua (bản nháp)

---

**Giải thuật 1:** Sinh đỉnh lưới cho một lần đường

---

**Input:** Đường gấp khúc  $P_0, \dots, P_{n-1} \in \mathbb{R}^3$ ; bề rộng làn  $w$

**Output:** Hai dãy đỉnh mép  $\{L_i\}, \{R_i\}$  và tập tam giác  $T$

$e_i \leftarrow \frac{P_{i+1} - P_i}{\|P_{i+1} - P_i\|}, \quad i = 0, \dots, n-2; //$  vector đơn vị mỗi đoạn

**for**  $i \leftarrow 0$  **to**  $n-1$  **do**

$d_i \leftarrow \begin{cases} e_0, & i = 0 \\ e_{n-2}, & i = n-1 \\ \frac{e_{i-1} + e_i}{\|e_{i-1} + e_i\|}, & 0 < i < n-1 \end{cases}; \quad //$  hướng (phân giác

tại đỉnh giữa)

$r_i \leftarrow \frac{d_i \times (-\hat{y})}{\|d_i \times (-\hat{y})\|}; \quad //$  pháp tuyến ngang

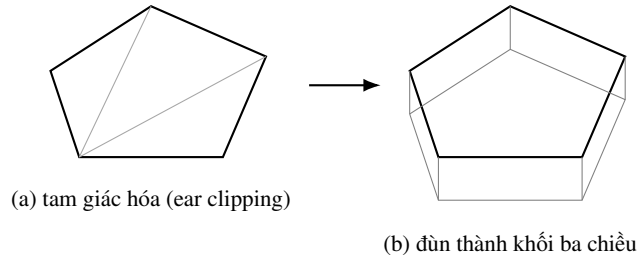
$\delta_i \leftarrow \begin{cases} w/2, & i \in \{0, n-1\} \\ \frac{w}{2 \cos \alpha_i}, & 0 < i < n-1 \end{cases} \quad \text{với } \cos \alpha_i = \sqrt{\frac{1+e_{i-1} \cdot e_i}{2}};$

$L_i \leftarrow P_i + \delta_i r_i, \quad R_i \leftarrow P_i - \delta_i r_i;$

$T \leftarrow \bigcup_{i=0}^{n-2} \{(L_i, R_i, L_{i+1}), (R_i, R_{i+1}, L_{i+1})\}; //$  mỗi đoạn  $\rightarrow$  2 tam giác

---

**Thuật toán dựng nút giao.** Đa giác nút giao được tam giác hóa bằng kỹ thuật cắt tai (ear clipping) chiếu trên mặt phẳng ngang, nhờ đó giữ được hình dạng lõm của nút. Trong trường hợp đa giác tự cắt do SUMO gộp nhiều nút sát nhau khiến cắt tai không hoàn tất, hệ thống tự chuyển sang phương án bao lồi (convex hull) để bảo đảm mặt nút vẫn được phủ kín thay vì bị rách. Mặt trên sau khi tam giác hóa được đùn xuống một đoạn để tạo thành khối ba chiều có mặt trên, mặt dưới và các mặt bên. Khối nút được gán một bộ va chạm dạng hộp bao để hệ vật lý hoạt động ổn định. Hình 0.2 minh họa quá trình tam giác hóa và đùn khối; Giải thuật 2 tóm tắt các bước.



**Hình 0.2:** Tam giác hóa đa giác nút giao rồi đùn thành khối ba chiều (bản nháp)

---

**Giải thuật 2: Dựng khối nút giao ba chiều**

---

**Input:** Đa giác nút giao  $V = (v_0, \dots, v_{m-1})$ ; vector độ cao  $\vec{h}$

**Output:** Tập tam giác lưới khối ba chiều  $\mathcal{T}$

$A(V) \leftarrow \frac{1}{2} \sum_{i=0}^{m-1} (x_i y_{i+1} - x_{i+1} y_i)$ ; // diện tích có dấu, chỉ số mod  $m$

**if**  $A(V) > 0$  **then**

$V \leftarrow (v_{m-1}, \dots, v_0)$ ; // đảo về thứ tự cùng chiều kim đồng hồ

$T \leftarrow \text{EarClip}(V)$ ; // tam giác hóa, giữ hình lõm

**if**  $|T| < m - 2$  **then**

$V \leftarrow \text{ConvexHull}(V)$ ; // đa giác tự cắt  $\rightarrow$  bao lồi  
 $T \leftarrow \text{EarClip}(V)$ ;

$T_{\text{trên}} \leftarrow T$ ;

$T_{\text{dưới}} \leftarrow \{ (c + \vec{h}, b + \vec{h}, a + \vec{h}) : (a, b, c) \in T \}$ ; // dời  $\vec{h}$ , đảo pháp tuyến

$T_{\text{bên}} \leftarrow \bigcup_{i=0}^{m-1} \{ (v_i, v_{i+1}, v_{i+1} + \vec{h}), (v_i, v_{i+1} + \vec{h}, v_i + \vec{h}) \}$ ;

$\mathcal{T} \leftarrow T_{\text{trên}} \cup T_{\text{dưới}} \cup T_{\text{bên}}$ ;

---

Hai thuật toán này dùng chung cho cả mạng sinh tự động và mạng nhập từ OpenStreetMap, do dữ liệu sau khi chuyển đổi đều ở cùng một định dạng mạng của SUMO. Nhờ đó, bản đồ thực tế cũng được dựng ba chiều bằng cùng một quy trình.

### 0.1.3 Kết quả đạt được

Hệ thống dựng được mạng đường ba chiều bám sát dữ liệu nguồn cho cả mạng sinh tự động và bản đồ thực tế từ OpenStreetMap. Mặt đường liền lạc tại khúc cua, nút giao được phủ kín kể cả với hình dạng phức tạp, và bề mặt va chạm cho phép xe di chuyển cùng đổi làn liên tục. Đây là nền tảng để hiển thị giao thông và để người dùng lái xe trong cảnh.

## 0.2 Tạo phương tiện của người dùng và chiếm quyền điều khiển

### 0.2.1 Vấn đề

Để mang lại góc nhìn của người tham gia giao thông, hệ thống cần cho phép người dùng điều khiển trực tiếp một phương tiện trong cảnh, gồm hai tình huống: tạo một phương tiện riêng của người dùng, và giành quyền điều khiển một phương tiện vốn do SUMO điều khiển. Khó khăn nằm ở chỗ một phương tiện do SUMO điều khiển được cập nhật vị trí theo dữ liệu mô phỏng và không phản ứng vật lý; muốn lái thủ công thì phải chuyển nó sang chuyển động theo vật lý, đồng thời phản

ánh vị trí mới về SUMO để các xe khác nhận biết.

### **0.2.2 Giải pháp**

Đồ án thiết kế một máy trạng thái xác định quyền điều khiển của mỗi phương tiện, trong đó quyền điều khiển và việc bật hay tắt vật lý được quyết định bởi trạng thái chứ không dựa vào sự hiện diện của thành phần. Khi người dùng chiếm quyền, phương tiện chuyển sang trạng thái do người dùng điều khiển: hệ thống bật thân cứng (Rigidbody) và các bộ va chạm bánh xe (Wheel Collider) để xe chuyển động theo vật lý, đồng thời ngừng cập nhật vị trí theo dữ liệu mô phỏng. Vị trí của xe do người dùng lái được gửi ngược về phía SUMO để cập nhật vào mô phỏng, giúp các phương tiện khác phản ứng theo. Khi không còn do người dùng điều khiển, phương tiện tắt phần vật lý và trở lại chế độ cập nhật theo dữ liệu để bảo đảm hiệu năng. Cách bật tắt vật lý theo trạng thái như vậy được gọi là cơ chế lai, chỉ trả chi phí vật lý cho đúng phương tiện đang được lái. Bên cạnh đó, hệ thống cho phép sinh một phương tiện riêng của người dùng, là phương tiện luôn do người dùng điều khiển và không bị SUMO chiếm lại.

### **0.2.3 Kết quả đạt được**

Người dùng có thể chiếm quyền một phương tiện bất kỳ đang chạy trong mô phỏng và lái nó bằng bàn phím với chuyển động có tính vật lý, hoặc sinh một phương tiện riêng để lái. Các phương tiện do SUMO điều khiển phản ứng với phương tiện người lái nhờ vị trí được phản ánh ngược về mô phỏng. Đây là tính năng tạo nên trải nghiệm theo góc nhìn người tham gia giao thông mà các công cụ quan sát thông thường không có.

## **0.3 Hệ thống va chạm giả lập tai nạn**

### **0.3.1 Vấn đề**

Khi người dùng lái xe trong dòng giao thông, một tình huống tự nhiên là va chạm với phương tiện khác. Hệ thống cần tái hiện hậu quả của va chạm một cách hợp lý: phương tiện bị tông phải dừng lại như một chiếc xe gặp tai nạn, gây ùn ứ cho các phương tiện phía sau, và sau một thời gian thì được dọn khỏi hiện trường. Thách thức là phương tiện bị tông vốn do SUMO điều khiển và không phản ứng vật lý, nên cần được chuyển trạng thái ngay tại thời điểm va chạm; ngoài ra việc dọn xe phải đáng tin cậy ngay cả khi đối tượng tạm thời bị ẩn khỏi cảnh.

### **0.3.2 Giải pháp**

Hệ thống phát hiện va chạm giữa phương tiện do người dùng điều khiển (hoặc xác xe) với phương tiện do SUMO điều khiển. Phương tiện bị tông được chuyển sang trạng thái xác xe: nó được bật vật lý để bị xô đẩy theo va chạm, đồng thời

phía SUMO đặt tốc độ về 0 và tiếp tục đồng bộ vị trí từ Unity, tạo vật cản gây ùn ứ cho dòng xe phía sau. Mỗi xác xe được dọn sau một số bước mô phỏng định trước. Để việc dọn xe đáng tin cậy, bộ đếm số bước được quản lý tập trung theo bước mô phỏng thay vì đặt trên từng đối tượng, nhờ đó không bị sai lệch khi đối tượng tạm bị ẩn để tối ưu hiển thị. Khi đủ số bước, phương tiện được chuyển sang trạng thái bị hủy và được thu hồi.

### **0.3.3 Kết quả đạt được**

Hệ thống tái hiện được tình huống tai nạn: xe bị tông dừng lại gây ùn ứ giống va chạm thực tế, sau đó tự được dọn khỏi hiện trường. Tính năng này làm phong phú các tình huống mà người dùng có thể quan sát và thử nghiệm, và là hệ quả tự nhiên của khả năng lái xe tương tác.

## **0.4 Lưu trữ kịch bản và phát lại**

### **0.4.1 Vấn đề**

Để phục vụ phân tích, đối chiếu và trình bày, hệ thống cần lưu lại diễn biến của mỗi lần chạy và cho phép xem lại mà không phải chạy lại mô phỏng. Nếu mỗi loại dữ liệu được ghi rời rạc thì khó truy vết theo từng lần chạy; ngoài ra, mô phỏng dài sinh khối lượng dữ liệu lớn nên không thể giữ toàn bộ trong bộ nhớ trước khi ghi.

### **0.4.2 Giải pháp**

Đồ án xây dựng cơ chế lưu trữ theo phiên, gom toàn bộ kết quả của một lần chạy vào một thư mục đặt theo tên bản đồ và thời điểm chạy, gồm nhật ký hành trình của xe, bản tóm tắt, dữ liệu mạng đường và chuỗi trạng thái theo từng bước. Chuỗi trạng thái được ghi theo cơ chế luồng với một vùng đệm: dữ liệu được ghi dần ra tệp khi vùng đệm đầy, nhờ đó không cần giữ toàn bộ trong bộ nhớ. Trên cơ sở chuỗi trạng thái đã lưu, hệ thống cung cấp chế độ phát lại đọc lại dữ liệu và tái hiện diễn biến trong Unity. Đáng chú ý, chế độ thời gian thực cũng ghi lại chuỗi trạng thái theo cùng định dạng, nên một lần chạy thời gian thực cũng có thể được phát lại về sau.

### **0.4.3 Kết quả đạt được**

Mỗi lần chạy được lưu trọn vẹn trong một thư mục phiên, thuận tiện cho truy vết và so sánh. Người dùng có thể phát lại một kịch bản đã lưu mà không cần chạy lại mô phỏng, phục vụ tốt cho việc phân tích và trình bày kết quả.

## **0.5 Giao tiếp thời gian thực tin cậy bằng tách khung bản tin**

### **0.5.1 Vấn đề**

Unity nhận dữ liệu trạng thái theo từng bước từ phía Python qua TCP. Vì TCP là giao thức hướng luồng, một lần nhận có thể chỉ chứa một phần bản tin hoặc chứa

nhiều bản tin gộp lại, dẫn tới lỗi khi phân tích dữ liệu. Hệ thống cần một cơ chế xác định ranh giới bản tin ổn định, đồng thời hỗ trợ cả bản tin dữ liệu lẫn bản tin điều khiển.

### **0.5.2 Giải pháp**

Mỗi bản tin được gửi kèm một dấu hiệu kết thúc đặt ở cuối. Phía nhận duy trì một vùng đệm cho mỗi kết nối, đọc thêm dữ liệu cho tới khi gặp dấu hiệu kết thúc, tách ra đúng một bản tin hoàn chỉnh và giữ lại phần dư cho lần sau. Cách này xử lý đúng bản chất luồng của TCP, tránh cả hai lỗi bản tin bị cắt và nhiều bản tin dính nhau. Trên nền giao thức này, hệ thống truyền song song bản tin dữ liệu trạng thái và bản tin điều khiển như tạm dừng, tiếp tục, kết thúc và cập nhật tham số.

### **0.5.3 Kết quả đạt được**

Kênh giao tiếp giữa Python và Unity hoạt động ổn định trong suốt quá trình mô phỏng thời gian thực, giảm lỗi phân tích dữ liệu và hỗ trợ điều khiển quá trình chạy một cách an toàn.

## **0.6 Tối ưu hiệu năng hiển thị quy mô lớn**

### **0.6.1 Vấn đề**

Hệ thống cần hiển thị đồng thời nhiều phương tiện trong khi vẫn duy trì tốc độ khung hình ổn định. Việc liên tục tạo và hủy đối tượng, cũng như vẽ toàn bộ đối tượng kể cả ngoài tầm quan sát, làm giảm hiệu năng đáng kể khi số lượng phương tiện tăng.

### **0.6.2 Giải pháp**

Hệ thống áp dụng một số kỹ thuật tối ưu. Thứ nhất là tái sử dụng đối tượng: phương tiện rời mạng không bị hủy mà được thu hồi để dùng lại cho phương tiện mới, giảm chi phí tạo và hủy. Thứ hai là lọc theo vùng quan sát: chỉ hiển thị các đối tượng trong tầm nhìn quanh camera và tạm ẩn các đối tượng ở ngoài, riêng phương tiện đang được người dùng điều khiển luôn được giữ hiển thị. Thứ ba là giảm số lệnh vẽ để hạ tải cho khâu kết xuất. Để chuyển động mượt khi tốc độ khung hình cao, vị trí phương tiện được nội suy giữa hai bước mô phỏng liên tiếp ở phía hiển thị.

### **0.6.3 Kết quả đạt được**

Nhờ các kỹ thuật trên, hệ thống duy trì được hiển thị ổn định khi số lượng phương tiện tăng, đồng thời chuyển động mượt mà ngay cả khi tốc độ khung hình cao hơn nhịp cập nhật của mô phỏng.

## **0.7 Kết chương**

Chương này đã trình bày các giải pháp và đóng góp nổi bật của đề án: hai thuật toán dựng đoạn đường và nút giao ba chiều bám sát dữ liệu, dùng chung cho mạng sinh tự động và bản đồ thực tế; cơ chế tạo phương tiện của người dùng và chiếm quyền điều khiển dựa trên máy trạng thái và bật tắt vật lý theo trạng thái; hệ thống va chạm giả lập tai nạn; cơ chế lưu trữ phiên và phát lại; giao tiếp thời gian thực tin cậy bằng tách khung bản tin; và các kỹ thuật tối ưu hiệu năng hiển thị. Các giải pháp này vừa giải quyết các vấn đề kỹ thuật cụ thể, vừa tạo nên những tính năng làm nên giá trị của hệ thống. Chương tiếp theo tổng kết kết quả đạt được và đề xuất hướng phát triển.